

SLF4J LOGGING FRAMEWORK

Exercise 1: Logging Error Messages and Warning Levels

Objective:

To use SLF4J with Logback to log messages at different severity levels (TRACE, DEBUG, INFO, WARN, ERROR) and observe how log level configuration filters which messages are displayed.

Code:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingDemo {

    private static final Logger logger =
        LoggerFactory.getLogger(LoggingDemo.class);

    public static void main(String[] args) {

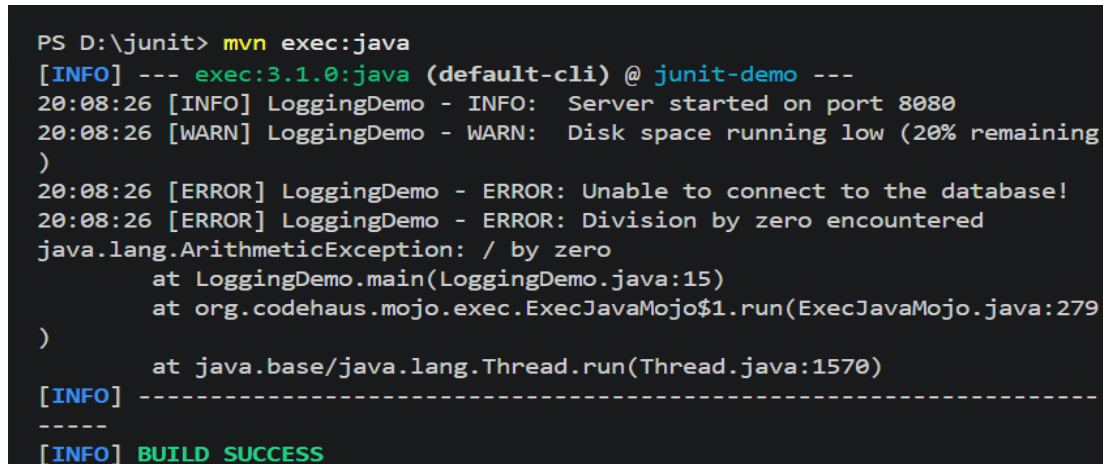
        logger.trace("TRACE: Entering main method");
        logger.debug("DEBUG: Application starting up");
        logger.info("INFO: Server started on port 8080");
        logger.warn("WARN: Disk space running low (20% remaining)");
        logger.error("ERROR: Unable to connect to the database!");

        // Simulating a caught exception logged at ERROR level
        try {
            int result = 10 / 0;
        } catch (ArithmeticException e) {
            logger.error("ERROR: Division by zero encountered", e);
        }
    }
}
```

logback.xml (configuration):

```
<!-- Change level to DEBUG or TRACE to see more output -->
<root level="INFO">
    <appender-ref ref="CONSOLE"/>
</root>
```

Program Output Screenshot:

A screenshot of a terminal window with a dark background and light-colored text. The text shows the execution of a Maven command to run a Java application. It includes several log messages from SLF4J, such as INFO, WARN, and ERROR, along with a stack trace for an ArithmeticException. The output ends with a BUILD SUCCESS message.

```
PS D:\junit> mvn exec:java
[INFO] --- exec:3.1.0:java (default-cli) @ junit-demo ---
20:08:26 [INFO] LoggingDemo - INFO: Server started on port 8080
20:08:26 [WARN] LoggingDemo - WARN: Disk space running low (20% remaining
)
20:08:26 [ERROR] LoggingDemo - ERROR: Unable to connect to the database!
20:08:26 [ERROR] LoggingDemo - ERROR: Division by zero encountered
java.lang.ArithmeticException: / by zero
    at LoggingDemo.main(LoggingDemo.java:15)
    at org.codehaus.mojo.exec.ExecJavaMojo$1.run(ExecJavaMojo.java:279
)
    at java.base/java.lang.Thread.run(Thread.java:1570)
[INFO] -----
-----
[INFO] BUILD SUCCESS
```

Output Text:

With root log level set to INFO, only INFO, WARN, and ERROR messages appear. TRACE and DEBUG messages are suppressed. The ArithmeticException stack trace is included with the ERROR log automatically.

Result:

SLF4J successfully logged messages at multiple levels. The Logback configuration controlled which levels were visible at runtime without changing application code.

Conclusion:

SLF4J provides a simple, unified logging API that is decoupled from the underlying logging framework (Logback, Log4j, etc.). Log levels (TRACE < DEBUG < INFO < WARN < ERROR) allow developers to control verbosity through configuration alone, making it easy to switch between verbose development logs and concise production logs.